



FINAL PROJECT (V3) · MLE-AI-201

Machine Learning

AI Engineering

AI Academy · National Artificial Intelligence Center

Released	Sun 04 Oct 2026	Points	100 (25% of course grade)
Due	Sun 18 Oct 2026, 23:59 + demo in the following session	Submit via	GitHub link + report PDF + slides PDF, on Moodle

1 Overview

This is the **team term project** for Machine Learning, build-it-yourself edition. Your team will **implement two core models from scratch in NumPy** — a **decision tree** and a **support-vector machine** — apply them to a real, messy **Azerbaijani real-estate dataset from bina.az**, and **benchmark them honestly against scikit-learn**. The goal is to understand these algorithms from the inside, to handle real data end to end, and to run a fair comparison, not to chase a single number.

Teams and the golden rule

Teams of 3–4 students. Form your team at kickoff and register it on Moodle. All members contribute to code *and* writing; the report names who did what.

Golden rule: your **decision tree** and your **SVM must be your own NumPy implementations**. You may use **scikit-learn** *only* as the baseline you compare against (its `DecisionTree*`, `SVC/SVR`, `SGDClassifier`) and for the optional bonuses. Using a library model as one of your two main models scores zero on the implementation criterion for that model.

2 Goals and learning objectives

By the end of this project your team should be able to:

- **Implement core ML algorithms from first principles.** Translate the maths from Weeks 1–5 (impurity and information gain; the SVM hinge-loss objective and the margin) into correct, tested NumPy code with a clean `fit/predict` API.
- **Turn raw, real-world data into a modelling dataset.** Clean a scraped listings dump (mixed types, Azerbaijani text, currency, missing values, outliers, duplicates), engineer sensible features, and build a leakage-free split.
- **Frame one dataset as two learning tasks.** A **regression** task (predict the sale price) and a derived **binary classification** task (price tier), and pick the right metric for each.
- **Run a fair, honest benchmark.** Compare your implementations against scikit-learn with

matched settings, report uncertainty, and explain the gaps (tie-breaking, float thresholds, optimiser differences).

- **Diagnose models, not just score them.** Read bias–variance behaviour from learning/validation curves, compute feature importance, and do real error analysis.
- **Work reproducibly and communicate clearly.** A seeded repo that runs from clean, a written report, and a defended demo.

3 The dataset: bina.az sale listings

Use the **Bina.az Sale Project dataset** (Kaggle, sehriyarmemmedli/binaaz-sale-project) — real property-sale listings scraped from **bina.az**, Azerbaijan’s largest real-estate platform. It is a genuine, uncleaned dump: expect mixed types, Azerbaijani column names and text, currency and unit quirks, missing values, duplicates, and outliers. **Cleaning it well is part of the project.**

Key columns (translate and sanity-check these yourself)

Column	Type	Meaning
price	numeric	Regression target — sale price (watch the currency).
Sahə (area)	numeric	Floor area in m ² .
Otaq sayı (rooms)	numeric	Number of rooms.
Mərtəbə (floor)	numeric	Floor number.
Torpaq sahəsi	numeric	Land area (houses/villas).
lat, lng	numeric	Coordinates — use for location features.
location, city	categorical	District / city.
Binanın növü	categorical	Building type (e.g. new build vs old stock).
Kateqoriya	categorical	Category.
Təmir (repair)	categorical	Repaired / not repaired.
İpoteka (mortgage)	categorical	Mortgage available.
Çıxarış	categorical	Bill of sale available.
description, attributes	text	Free text (optional feature engineering).

Leakage: read before you pick features

Columns such as `unit_price` and `total_price` are **arithmetic functions of the target** (price per m², price restated). Feeding them into a **price** model is **target leakage** — your validation score will look spectacular and the model will be useless in deployment. **Drop them** from the price task (and justify it in the report). Likewise, remove identifiers (`owner_name`, `shop_name`, raw address) and anything that encodes the answer. This is exactly the leakage lesson from Weeks 1 and 3; we will check for it.

3.1 Two tasks from one dataset

- **Task A — Regression (price).** Predict price (consider modelling $\log(\text{price})$, since prices are heavy-tailed). This is the main task for your **decision tree regressor**.
- **Task B — Classification (price tier).** Derive a binary label from the *cleaned* price — e.g. “**premium**” vs “**standard**” by splitting at the **median** (or a documented percentile). This is the task for your **from-scratch SVM** and your tree in classification mode. Define the threshold *before* modelling and from the training split only — do not let the test set set the threshold.

4 Milestones (two-week project)

- **Day 1 — Kickoff.** Team registered; dataset downloaded and loaded; first EDA; both tasks framed; leakage columns identified.
- **End of Week 1 — Working models.** A tree that fits and predicts and an SVM that trains (loss decreasing), both passing a sanity test against sklearn on a toy split, running on your cleaned data. Bring blockers to office hours.
- **End of Week 2 — Final submission + demo.** Full comparison, report, runnable repo, and a short presentation defended in the session after the deadline.

5 Tasks

Points sum to 100; each bonus adds up to +10 (capped per course policy).

Problem 1 Data, cleaning, EDA and framing

[15 pts]

Load the bina.az dump and turn it into a modelling dataset.

- Clean it.** Handle currency, type coercion (numeric columns arriving as strings), missing values, duplicates, and outliers; document every decision. Translate/standardise the Azerbaijani column names you keep.
- Explore.** Price distribution (and $\log(\text{price})$), area vs price, rooms, location effects (use lat/lng), class balance of your price-tier label.
- Frame & split.** State both tasks and their metrics, remove the leakage columns above, encode categoricals, and build a **stratified** (for Task B) **train/validation/test split**. Report the split sizes.

Problem 2 Implement a decision tree from scratch

[25 pts]

Implement a decision-tree in NumPy with a clean `fit/predict` API (mirror sklearn's) that supports **both tasks**.

- Split criteria.** For classification implement **Gini** and **entropy** with information gain; for regression implement **variance / MSE reduction**:

$$G = 1 - \sum_k p_k^2, \quad H = - \sum_k p_k \log_2 p_k, \quad \text{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c), \quad I_{\text{reg}}(\text{node}) = \frac{1}{N} \sum_i (y_i - \bar{y})^2.$$

The criterion is a hyperparameter; the split search picks the feature/threshold with the largest gain. Implement **threshold splits** for numeric features (sort values, test candidate thresholds).

- Growth & stopping.** Support at least `max_depth`, `min_samples_split`, `min_samples_leaf`, and `min_impurity_decrease`. Each must actually change the tree.
- Correctness.** Recursive build; leaf prediction by **majority class** (classification) or **mean** (regression); `predict_proba` from leaf frequencies. Include a test that your tree matches sklearn's splits on a tiny, fully-determined example.

Keep it readable and commented.

Problem 3 Implement a support-vector machine from scratch

[20 pts]

Implement a **linear soft-margin SVM** for Task B in NumPy — **no quadratic-programming or SMO solver required**. Train it by **sub-gradient descent on the regularised hinge-loss primal** (the Pegasos method); see the box below.

- (a) **Objective & solver.** Minimise $J(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$ over labels $y_i \in \{-1, +1\}$, using the (stochastic) sub-gradient update below. Track the objective and show it decreasing.
- (b) **Hyperparameters.** Expose the regularisation λ (equivalently C), the number of epochs, and the learning-rate schedule; show each affects the margin / decision boundary. **Scale your features first** — the hinge loss is scale-sensitive.
- (c) **A margin you can see.** Report the learned $\|\mathbf{w}\|$ and margin width $2 / \|\mathbf{w}\|$, and identify which training points are the support vectors (those with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 1$).
- (d) **Non-linearity (choose one).** Either (i) **kernelised Pegasos** maintaining per-example coefficients with an RBF kernel, or (ii) **random Fourier features** mapping $\mathbf{x}$ to a higher-dimensional space and then running your *linear* solver. State which and why.

How to write an SVM without a QP solver (Pegasos)

You do **not** need to solve the dual QP. The soft-margin SVM is just regularised hinge-loss minimisation, and sub-gradient descent handles it in a few lines. Set learning rate $\eta_t = \frac{1}{\lambda t}$ at step t ; pick a random example i (augment $\mathbf{x}$ with a constant 1 to fold in the bias):

$$\text{if } y_i(\mathbf{w}^\top \mathbf{x}_i) < 1 : \mathbf{w} \leftarrow (1 - \eta_t \lambda) \mathbf{w} + \eta_t y_i \mathbf{x}_i, \quad \text{else : } \mathbf{w} \leftarrow (1 - \eta_t \lambda) \mathbf{w}.$$

That update is the solver. A full-batch version (average the sub-gradient over all points each epoch) is even simpler to reason about and fine for this dataset. For a kernel, keep coefficients α_i instead of $\mathbf{w}$ and replace inner products with $K(\mathbf{x}_i, \mathbf{x}_j)$; or approximate the RBF kernel with random Fourier features and keep the linear update. Reference: Shalev-Shwartz et al., *Pegasos*, 2011.

Problem 4 Train, tune and study your models

[15 pts]

- (a) **Tree.** Compare **Gini vs entropy** (Task B) and sweep `max_depth` (plus one more hyperparameter); show the **overfitting / bias-variance** story with validation or learning curves.
- (b) **SVM.** Sweep λ/C (and the kernel bandwidth if used); show how the margin and validation metric trade off; confirm the training objective converges.
- (c) **Final configs.** Pick a final configuration for each from the **validation** evidence only — no test-set peeking.

Problem 5 Benchmark against scikit-learn

[10 pts]

Compare, with *matched* hyperparameters where possible:

- (a) **Tree** vs `DecisionTreeRegressor` (Task A) and `DecisionTreeClassifier` (Task B): comparable structure/predictions? Explain gaps (tie-breaking, float thresholds).
- (b) **SVM** vs `SVC` / `SGDClassifier` (hinge loss), features scaled inside a `Pipeline`.
- (c) **Report** test-set metrics with uncertainty (cross-validated mean $\pm$ std) and a note on **fit/predict time** and scaling.

Problem 6 Evaluation, metrics and error analysis

[10 pts]

Report **task-appropriate metrics**: for **Task A** RMSE, MAE and R^2 (on price and/or $\log(\text{price})$); for **Task B** precision/recall/F1 and ROC-AUC or PR-AUC with the **confusion matrix**. Use your tree's **impurity-decrease feature importances**, inspect the **SVM weights**, and analyse **where and why** each model fails (e.g. luxury outliers, sparse districts). A results table comparing all models on both tasks is required.

Problem 7 Reproducibility and communication

[5 pts]

A version-controlled repo that runs from clean (pinned deps, fixed seeds, clear README, one command/notebook that reproduces every number and figure), plus the completed `TEMPLATE_Report_AIEng` PDF and a short demo. Figures labelled, claims tied to code, contributions named.

Problem 8 From-scratch linear baselines

[Bonus +10 pts]

Optional. Implement **linear/Ridge regression from scratch** via the normal equations $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ (the derivation you did on the final) as a Task A baseline, and/or a from-scratch **SVR** using sub-gradient descent on the ϵ -insensitive loss. Compare against your tree and sklearn.

Problem 9 Ensembles from your own trees (Weeks 5 to 6)

[Bonus +10 pts]

Optional, after the core is solid. Using **your own** trees as weak learners, build a **random forest** (bootstrap + feature subsampling) or a small **AdaBoost** / **gradient-boosting** loop from scratch; show test error dropping as learners are added, and compare against sklearn's `RandomForest` / `GradientBoosting`.

Problem 10 Unsupervised and geospatial EDA

[Bonus +10 pts]

Optional. Use **PCA** and **k-means** (or DBSCAN on `lat/lon`) to find market segments or neighbourhood clusters, and use them to sharpen your error analysis.

6 Deliverables

The project **starts now and is due in two weeks** (released Sun 04 Oct 2026; due **Sun 18 Oct 2026, 23:59, Baku**). Submit **three things on Moodle; one member submits on behalf of the team** (others do not upload duplicates):

1. **GitHub link** to your repository, tagged `v1.0-final`: your from-scratch tree and SVM, the data pipeline, a pinned `requirements.txt`, fixed seeds, and a `README.md` with **one-command reproduction** of every headline number and figure, plus how to obtain the dataset (do not commit the raw archive). Keep the `contribution_report` in the repo root.
2. **Report PDF** (IEEE two-column; use the self-contained template in the starter pack): problem framing, cleaning decisions, both algorithms described, experiments, the required results table(s), error analysis, honest limitations, and an AI-assistance disclosure.
3. **Slides PDF** sized for a short presentation, defended in the session after the deadline.

7 Grading rubric

Criterion	Weight	What “excellent” looks like
From-scratch implementations	35 %	Correct tree (Gini/entropy/MSE + split search + working hyperparameters) <i>and</i> SVM (sub-gradient hinge solver; margin, support vectors); clean, tested <code>fit/predict</code>
Data & experimental design	25 %	Real cleaning done and justified; leakage removed; fair matched comparison vs sklearn; tuning without test-set leakage; speed + accuracy reported
Evaluation & analysis	20 %	Right metrics per task with uncertainty; results table; real feature-importance / weight and error analysis
Reproducibility	10 %	Versioned, seeded repo that runs from clean and reproduces every result
Communication	10 %	Clear report + demo; evidenced claims; honest limitations; contributions named

Core totals 100%; each bonus can add up to +10% (capped per course policy).

8 Penalties and deductions

Automatic deductions

- **Library model as a core model.** Using scikit-learn (or any library) as your tree or your SVM instead of your own code: that model’s implementation marks are **zero**.
- **Target leakage.** Training the price model on `unit_price/total_price` or other target-derived columns.
- **Test-set leakage / tuning on test.** Choosing the price-tier threshold, hyperparameters, or

any preprocessing statistic from the test set.

- **Non-reproducible headline result** / no one-command entry point / unseeded randomness.
- **Leftover template text** in the report, or a missing results table.
- **Missing demo slides, or missing/late contribution report.**

Individual penalties (learned from past projects)

- **Under-contribution.** A member who clearly carried little of the work — by Git history, the contribution report, or the demo — gets an **individually reduced grade**. The penalty hits the person, not the team.
- **Free-riding.** Repeatedly missing team sessions, ignoring team communication, or leaving your module to the last day risks an individual penalty regardless of the team's score.
- **Last-minute dumping.** Code committed hours before the deadline that was never integrated, reviewed, or understood does not count as a contribution.
- **Misreported contributions.** Claims that contradict Git history or the demo count as an academic-integrity issue.
- **Cannot defend your part.** At the demo every member must explain any part of the work; repeated “I don't know” answers about your listed contributions lower your individual score.

9 Documentation and reference links

Build the two models yourself; use these to compare and to understand the reference behaviour.

- [Decision trees \(User Guide\)](#) • [DecisionTreeClassifier](#) • [DecisionTreeRegressor](#)
- [SVM](#) • [SVC](#) • [SVR](#) • [SGDClassifier](#) • [Pipeline](#) • [Preprocessing / scaling](#)
- [Metrics & scoring](#) • [Cross-validation](#) • [Tuning / grid search](#)
- Bonus: [RandomForest](#) • [GradientBoosting](#) • [PCA](#) • [KMeans](#)

10 Rules

- The decision tree, the SVM, and any from-scratch regression/ensemble must be your team's own NumPy code. scikit-learn is for the *comparison* baselines and bonuses only.
- Teamwork within your registered team only. Cite every external resource, snippet, dataset, and AI-assistant use; undisclosed copying is plagiarism (syllabus §8.5).
- Respect the dataset's licence and bina.az's terms; do not re-scrape or redistribute personal data from the listings.
- The final project **cannot be submitted late** (syllabus §8.4); request any accommodation, with documentation, before the deadline.

Tip. Get each model working end to end on a tiny clean subset first (tree: one criterion + `max_depth`; SVM: linear, full-batch sub-gradient), validate against sklearn on a toy example, *then* add criteria, hyperparameters, and the kernel. A correct small model beats a broken ambitious one.